

# PRACTICAL - 05

---

**Name:** Hansika Reddy Gunapati

**Rollno:** 2520030237

**Section:** 8

---

## Experiment No. 5

**Title:** Implementation of a Producer-Consumer Communication System and a Shell Pipeline Emulator using Anonymous Pipes, fork(), dup2(), and exec() System Calls.

### Aim:

To develop C programs that (i) implement a producer-consumer system using anonymous pipes where a parent process produces data and a child process consumes it, measuring the communication efficiency, and (ii) emulate the shell command `ls -l | grep ".c"` using fork(), pipe(), dup2(), and exec() system calls.

### Objective:

- To understand inter-process communication (IPC) using anonymous pipes.
- To implement a producer-consumer relationship between a parent and child process.
- To measure and evaluate the communication efficiency (throughput) of pipe-based IPC.
- To understand file descriptor duplication using dup2().
- To emulate shell command piping between two processes using pipe(), fork(), dup2(), and exec().

### Theory:

An anonymous pipe is a unidirectional inter-process communication mechanism provided by the operating system that allows related processes (such as a parent and its child created via fork()) to exchange data. A pipe has two ends represented by file descriptors: fd[0] for reading and fd[1] for writing. Data written to the write end can be read sequentially from the read end, and the pipe acts as a bounded, in-kernel buffer.

In a producer-consumer system, the producer (parent process) writes data into the pipe, while the consumer (child process) reads and processes this data. Since the pipe has a limited buffer size, the OS automatically blocks the producer when the pipe is full and blocks the consumer when the pipe is empty, providing implicit synchronization.

The dup2() system call is used to duplicate a file descriptor onto another descriptor number, commonly used to redirect standard input/output to a pipe. This is exactly how the shell implements command pipelines such as `ls -l | grep ".c"`: the standard output of the first command is redirected to the write end of a pipe, and the

standard input of the second command is redirected to the read end of the same pipe, after which each command is executed using the `exec()` family of system calls.

Communication efficiency in this context is measured as throughput — the amount of data transferred through the pipe per unit time — calculated by timing the total duration of the write operations against the total volume of data transferred.

#### Algorithm (Part 1 – Producer-Consumer):

- Start the program and create a pipe using `pipe()`.
- Create a child process using `fork()`.
- In the child (consumer): close the unused write end, read data from the pipe in a loop until EOF, and record the total bytes received.
- In the parent (producer): close the unused read end, note the start time, write a fixed number of data chunks into the pipe, close the write end to signal EOF, and wait for the child.
- Note the end time in the parent and compute elapsed time and throughput.
- Display the number of bytes transferred and the communication efficiency.
- Stop.

#### Algorithm (Part 2 – `ls -l | grep ".c"` Emulation):

- Start the program and create a pipe using `pipe()`.
- Create the first child process using `fork()`.
- In the first child: redirect stdout to the pipe's write end using `dup2()`, close both pipe descriptors, and execute `execlp("ls", "ls", "-l", NULL)`.
- Create the second child process using `fork()`.
- In the second child: redirect stdin to the pipe's read end using `dup2()`, close both pipe descriptors, and execute `execlp("grep", "grep", ".c", NULL)`.
- In the parent: close both ends of the pipe (so no descriptor keeps it open) and wait for both children using `waitpid()`.
- Display a completion message.
- Stop.

#### System Calls Used:

| System Call                                  | Purpose                                                                                     |
|----------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>pipe()</code>                          | Creates a unidirectional communication channel (read and write file descriptors)            |
| <code>fork()</code>                          | Creates a child process                                                                     |
| <code>dup2()</code>                          | Duplicates a file descriptor onto stdin/stdout for redirection                              |
| <code>exec()</code> family                   | Replaces a process image to run an external command ( <code>ls</code> , <code>grep</code> ) |
| <code>read()</code> / <code>write()</code>   | Transfers data through the pipe                                                             |
| <code>wait()</code> / <code>waitpid()</code> | Synchronizes parent and child process completion                                            |
| <code>close()</code>                         | Closes unused pipe file descriptors                                                         |

#### Program 1: Producer-Consumer with Communication Efficiency Measurement

File: `producer_consumer.c`

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
#include <sys/time.h>

#define CHUNK_SIZE 64
#define NUM_CHUNKS 1000

int main() {
    int fd[2];
    pid_t pid;
    struct timeval start, end;

    if (pipe(fd) < 0) {
        perror("pipe failed");
        exit(1);
    }

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        /* ---- Child Process : Consumer ---- */
        char buffer[CHUNK_SIZE];
        int total_bytes = 0;
        close(fd[1]); /* close write end, child only reads */

        int n;
        while ((n = read(fd[0], buffer, CHUNK_SIZE)) > 0) {
            total_bytes += n;
        }

        printf("\nChild (Consumer) Process\n");
        printf("Child PID      : %d\n", getpid());
        printf("Total bytes read: %d\n", total_bytes);

        close(fd[0]);
        exit(0);
    } else {
        /* ---- Parent Process : Producer ---- */
        char data[CHUNK_SIZE];
        memset(data, 'A', CHUNK_SIZE);
        close(fd[0]); /* close read end, parent only writes */

        gettimeofday(&start, NULL);
        for (int i = 0; i < NUM_CHUNKS; i++) {
            write(fd[1], data, CHUNK_SIZE);
        }
        close(fd[1]); /* signals EOF to child */

        wait(NULL);
        gettimeofday(&end, NULL);

        double elapsed = (end.tv_sec - start.tv_sec) +
            (end.tv_usec - start.tv_usec) / 1e6;
    }
}

```

```
    long total_sent = (long)CHUNK_SIZE * NUM_CHUNKS;
    double throughput = total_sent / elapsed / 1024.0; /* KB/s */

    printf("\nParent (Producer) Process\n");
    printf("Parent PID      : %d\n", getpid());
    printf("Total bytes sent   : %ld\n", total_sent);
    printf("Time taken          : %f seconds\n", elapsed);
    printf("Communication efficiency (throughput): %.2f KB/s\n", throughput);
}
return 0;
}
```

**Sample Output (Program 1):**

```
teja@Hasini:~$ nano producer_consumer.c
teja@Hasini:~$ gcc producer_consumer.c -o producer_consumer
teja@Hasini:~$ ./producer_consumer
```

```
Child (Consumer) Process
Child PID      : 924
Total bytes read : 64000
```

```
Parent (Producer) Process
Parent PID     : 923
Total bytes sent : 64000
Time taken     : 0.000930 seconds
Communication efficiency (throughput): 67204.30 KB/s
```

```
teja@Hasini:~$ wsl
Command 'wsl' not found, but can be installed with:
sudo apt install wsl
teja@Hasini:~$ ./producer_consumer
```

```
Child (Consumer) Process
Child PID      : 953
Total bytes read : 64000
```

```
Parent (Producer) Process
Parent PID     : 952
Total bytes sent : 64000
Time taken     : 0.000461 seconds
Communication efficiency (throughput): 135574.84 KB/s
```

```
teja@Hasini:~$ ./producer_consumer
```

```
Child (Consumer) Process
Child PID      : 964
Total bytes read : 64000
```

```
Parent (Producer) Process
Parent PID     : 963
Total bytes sent : 64000
Time taken     : 0.000673 seconds
```

## Program 2: Emulating ls -l | grep ".c" using pipe(), fork(), dup2(), exec()

File: pipeline.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main() {
    int fd[2];
```

```

pid_t pid1, pid2;

if (pipe(fd) < 0) {
    perror("pipe failed");
    exit(1);
}

/* ---- First child : executes ls -l ---- */
pid1 = fork();
if (pid1 < 0) {
    perror("fork failed");
    exit(1);
}
if (pid1 == 0) {
    dup2(fd[1], STDOUT_FILENO); /* redirect stdout to pipe write end */
    close(fd[0]);
    close(fd[1]);
    execlp("ls", "ls", "-l", NULL);
    perror("exec ls failed");
    exit(1);
}

/* ---- Second child : executes grep ".c" ---- */
pid2 = fork();
if (pid2 < 0) {
    perror("fork failed");
    exit(1);
}
if (pid2 == 0) {
    dup2(fd[0], STDIN_FILENO); /* redirect stdin from pipe read end */
    close(fd[0]);
    close(fd[1]);
    execlp("grep", "grep", ".c", NULL);
    perror("exec grep failed");
    exit(1);
}

/* ---- Parent : closes both ends and waits ---- */
close(fd[0]);
close(fd[1]);
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("\nPipeline execution completed (ls -l | grep \".c\").\n");
return 0;
}

```

**Sample Output (Program 2):**

```

teja@Hasini:~$ nano pipeline.c
teja@Hasini:~$ nano pipeline.c
teja@Hasini:~$ gcc pipeline.c -o pipeline
teja@Hasini:~$ ./pipeline
-rw----- 1 teja teja 1034 Aug  8 10:06 filecopy.c.save
-rw----- 1 teja teja 1180 Aug  8 10:07 filecopy.c.save.1
-rw-r--r-- 1 teja teja  471 Aug 18 06:04 fork.c
-rw-r--r-- 1 teja teja 1180 Aug  8 10:08 hello.c
-rw-r--r-- 1 teja teja  896 Jul 30 04:56 os_task.c
-rw-r--r-- 1 teja teja 3258 Aug  8 10:00 parser.c
-rw-r--r-- 1 teja teja 2595 Aug 27 05:44 path_resolver.c
-rw-r--r-- 1 teja teja 1218 Sep 17 05:03 pipeline.c
-rwxr-xr-x 1 teja teja 16528 Sep 17 05:01 producer_consumer
-rw-r--r-- 1 teja teja 2084 Sep 17 05:01 producer_consumer.c
-rw----- 1 teja teja 2084 Sep 17 05:00 producer_consumer.c.sa
ve
-rw----- 1 teja teja 2084 Sep 17 05:01 producer_consumer.c.sa
ve.1
-rw-r--r-- 1 teja teja 7352 Sep 17 03:31 question.c
-rw-r--r-- 1 teja teja 2181 Sep 17 03:13 question7.c
-rw-r--r-- 1 teja teja 4530 Sep 17 03:21 question9.c
-rw-r--r-- 1 teja teja 3576 Aug 25 03:49 quotes.c
-rwxr-xr-x 1 teja teja 21632 Sep 17 03:42 redirection
-rw-r--r-- 1 teja teja 7814 Sep 17 03:42 redirection.c
-rwxr-xr-x 1 teja teja 21696 Sep 17 03:51 redirection_pipeline
-rw-r--r-- 1 teja teja 18366 Sep 17 03:51 redirection_pipeline.c

```

Pipeline execution completed (ls -l | grep ".c").

```

teja@Hasini:~$ touch abc.c
touch program.c
touch hello.txt
touch sample.py
teja@Hasini:~$ ./pipeline
-rw-r--r-- 1 teja teja    0 Sep 17 05:04 abc.c
-rw----- 1 teja teja 1034 Aug  8 10:06 filecopy.c.save
-rw----- 1 teja teja 1180 Aug  8 10:07 filecopy.c.save.1
-rw-r--r-- 1 teja teja  471 Aug 18 06:04 fork.c

```

```

teja@Hasini:~$ ./pipeline
-rw-r--r-- 1 teja teja      0 Sep 17 05:04 abc.c
-rw----- 1 teja teja  1034 Aug  8 10:06 filecopy.c.save
-rw----- 1 teja teja  1180 Aug  8 10:07 filecopy.c.save.1
-rw-r--r-- 1 teja teja   471 Aug 18 06:04 fork.c
-rw-r--r-- 1 teja teja  1180 Aug  8 10:08 hello.c
-rw-r--r-- 1 teja teja   896 Jul 30 04:56 os_task.c
-rw-r--r-- 1 teja teja  3258 Aug  8 10:00 parser.c
-rw-r--r-- 1 teja teja  2595 Aug 27 05:44 path_resolver.c
-rw-r--r-- 1 teja teja  1218 Sep 17 05:03 pipeline.c
-rwxr-xr-x 1 teja teja 16528 Sep 17 05:01 producer_consumer
-rw-r--r-- 1 teja teja  2084 Sep 17 05:01 producer_consumer.c
-rw----- 1 teja teja  2084 Sep 17 05:00 producer_consumer.c.sa
ve
-rw----- 1 teja teja  2084 Sep 17 05:01 producer_consumer.c.sa
ve.1
-rw-r--r-- 1 teja teja      0 Sep 17 05:04 program.c
-rw-r--r-- 1 teja teja  7352 Sep 17 03:31 question.c
-rw-r--r-- 1 teja teja  2181 Sep 17 03:13 question7.c
-rw-r--r-- 1 teja teja  4530 Sep 17 03:21 question9.c
-rw-r--r-- 1 teja teja  3576 Aug 25 03:49 quotes.c
-rwxr-xr-x 1 teja teja 21632 Sep 17 03:42 redirection
-rw-r--r-- 1 teja teja  7814 Sep 17 03:42 redirection.c
-rwxr-xr-x 1 teja teja 21696 Sep 17 03:51 redirection_pipeline
-rw-r--r-- 1 teja teja 18366 Sep 17 03:51 redirection_pipeline.c

Pipeline execution completed (ls -l | grep ".c").
teja@Hasini:~$ ls -l | grep ".c"
-rw-r--r-- 1 teja teja      0 Sep 17 05:04 abc.c
-rw----- 1 teja teja  1034 Aug  8 10:06 filecopy.c.save
-rw----- 1 teja teja  1180 Aug  8 10:07 filecopy.c.save.1
-rw-r--r-- 1 teja teja   471 Aug 18 06:04 fork.c
-rw-r--r-- 1 teja teja  1180 Aug  8 10:08 hello.c
-rw-r--r-- 1 teja teja   896 Jul 30 04:56 os_task.c
-rw-r--r-- 1 teja teja  3258 Aug  8 10:00 parser.c

```

#### Observation:

- A pipe was successfully created and used to transfer data between a parent (producer) and child (consumer) process.
- The operating system's pipe buffer provided implicit synchronization between the producer and consumer without needing explicit locks.
- The communication efficiency (throughput) was computed by timing the write operations against the volume of data transferred through the pipe.
- Using `dup2()`, standard output of one process and standard input of another were successfully redirected to the two ends of a pipe, allowing `exec()`-launched programs (`ls` and `grep`) to communicate exactly as the shell pipeline `ls -l | grep ".c"` would.
- Closing unused pipe ends in every process (including the parent) was essential — otherwise the reading process never receives EOF and blocks indefinitely.

**Result:**

The programs were successfully executed. A producer-consumer communication system was implemented using anonymous pipes, and the communication efficiency (throughput) was measured. The shell command `ls -l | grep ".c"` was successfully emulated using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls, correctly filtering and displaying only the ".c" files from the directory listing.